CP317A - Group Project

Sprint 01: Implementation and Initial Prototype

Group 22

Drive Now Car Rentals

Car Rental Management System

Group Members

Albina Qureshi Backend Developer

Aryan Kainth Scrum Master

Dheeraj Bashal Product Owner

Sadaf Faizi Frontend Developer

2. Sprint Goal

Sprint goal

Deliver the first working prototype of Drive Now Car Rentals, built with Python (a Tkinter interface) and a MySQL database. Customers can register an account, sign in securely, and browse, search, and view details for available cars. Administrators sign in through a separate role to add, update, and remove cars from the inventory.

Stories selected for Sprint 1

Our Milestone 02 backlog has 17 user stories. We chose six of them for this sprint, worth 25 story points in total: ACC-01 (Manage Account), ACC-02 (Sign In and Role Access), CAR-01 (Browse Cars), CAR-02 (Search Cars), CAR-03 (View Car Details), and ADMIN-01 (Manage Cars). The browse, search, and car details screens follow Wireframe 1 (Browse and Search) and Wireframe 2 (Car Details and Booking) from Milestone 02. Customers can search by pick-up and return dates, car type, price range, and make or model, and each car's year is shown on its card and details page.

Why these stories were prioritized

These six stories are the foundation of the system, and the rest of the backlog depends on them. Customers cannot book a car without an account, so registration and sign-in come first. Sign-in also sets up the role-based access control from our ethical considerations, which keeps customers out of the administrator tools. We built the matching security measures in from the start: passwords are stored hashed, a wrong email and a wrong password give the same "Invalid email or password" message, and administrator functions are checked on the backend. Public registration only creates customer accounts, and administrator accounts are created by the team.

ADMIN-01 is just as early because the system starts with no cars in it. Until an administrator can add vehicles, there is nothing for customers to browse or search. Removed cars are marked as removed instead of deleted, so they are hidden from customers while their records are kept. Once cars exist, CAR-01, CAR-02, and CAR-03 let customers find a vehicle and review its details.

We left booking, cancelling, and payments for Sprint 2 on purpose. As planned in Milestone 02, they are the most complex features in the system because they have to prevent double bookings and handle payment information securely, and they can only be built once accounts and the car inventory are working. Because there are no real bookings yet, the Book Now button on the car details page is a placeholder. By the end of Sprint 1, a customer can register, sign in, and find a car, and an administrator can keep the inventory up to date. We planned about 58 hours of work across the four team members.

3. Sprint Backlog

Story ID and Title	Pts	Priority	Assigned	Tasks (hours)	Status
ACC-01 Manage Account	5	High	Dheeraj, Sadaf, Albina, Aryan	ACC-01.1 Plan account data and validation (2) ACC-01.2 Registration form (3) ACC-01.3 Implement account creation (4) ACC-01.4 Secure password storage and test registration (3)	In Progress (profile editing not built, see Section 5)
ACC-02 Sign In and Role Access	5	High	Dheeraj, Sadaf, Albina, Aryan	ACC-02.1 Define authentication and roles (2) ACC-02.2 Sign-in and sign-out UI (2) ACC-02.3 Authentication and role-based access (4) ACC-02.4 Test access restrictions (2)	Done
CAR-01 Browse Cars	3	High	Dheeraj, Sadaf, Albina, Aryan	CAR-01.1 Define car listing data and card layout (1) CAR-01.2 Browse cars page (3) CAR-01.3 Active car listing data (2) CAR-01.4 Testing	In Progress (seats and photo not shown, see Section 5)
CAR-02 Search Cars	5	High	Dheeraj, Sadaf, Albina, Aryan	CAR-02.1 Search form and filters (3) CAR-02.2 Validate dates and filter inputs (2) CAR-02.3 Search and filter query (4) CAR-02.4 Testing	Done
CAR-03 View Car Details	2	Medium	Dheeraj, Sadaf, Albina, Aryan	CAR-03.1 Car details view (2) CAR-03.2 Connect details view to the selected car (2) CAR-03.3 Testing	Done
ADMIN-01 Manage Cars	5	High	Dheeraj, Sadaf, Albina, Aryan	ADMIN-01.1 Define inventory fields and admin rules (2) ADMIN-01.2 Admin car management UI (3) ADMIN-01.3 Car create, update, and remove (4) ADMIN-01.4 Protect and test admin functions (2)	Done

Total: 6 stories, 25 story points. Project setup (SETUP.1: repository, README, .gitignore, sample data) supports all six stories. The three testing tasks (CAR-01.4, CAR-02.4, CAR-03.3).

4. Implementation Progress

What was built. The prototype is a Python desktop application with a Tkinter interface that follows the wireframes. The data for our project is stored in a MySQL database, data such as users, locations, cars, reservations, and payments tables. This data will be accessed through SQLAlchemy. The interface files (app_ui.py, browse_ui.py, admin_ui.py) call a separate logic layer (account_service.py, auth_service.py, car_listing.py, car_search.py, car_admin.py), so the backend rules are checked in one place no matter which screen asks. In Sprint 1 we built account registration, sign-in and sign-out with role-based access, the browse page, search with filters, the car details page, and the administrator page for adding, updating, and removing cars.



Figure 1. Sign In screen (ACC-02).
window (ACC-01).



Figure 2. Create Account form filled in, next to the Sign In

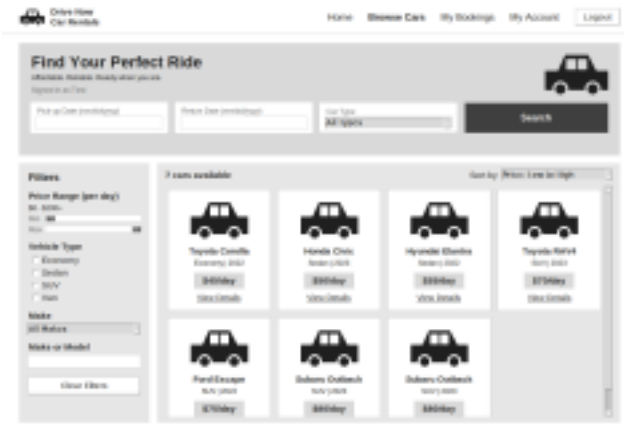


Figure 3. Browse Cars page with the search bar and filters (CAR-01).



Figure 4. Search for 11/22 to 11/25/2026. The Corolla is booked 11/20 to 11/23, so it is left out (CAR-02).

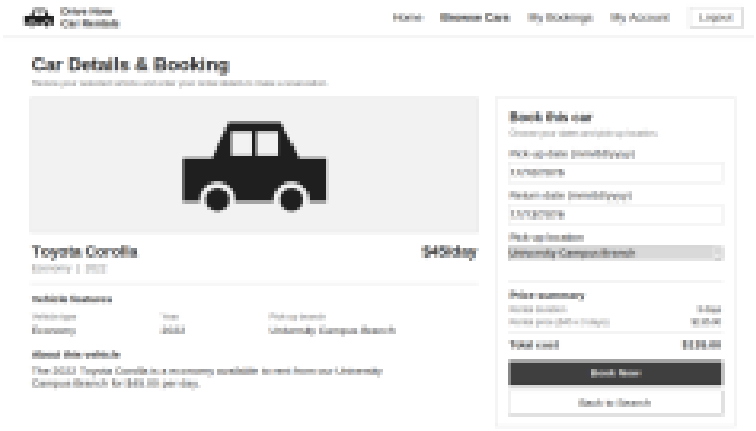




Figure 5. (The one on top) Car Details and Booking page for the Toyota Corolla (CAR-03)
Figure 6. (The one at the bottom) Administrator page after adding a 2024 Tesla Model 3 (ADMIN-01).

Code snippets

Availability check in car_search.py. Two bookings overlap when each one starts on or before the day the other ends, so cars booked on overlapping dates are left out of the results:

```
if pickup_date and return_date:
 booked_cars = session.query(Reservation.car_id).filter(
 Reservation.status.in_(BLOCKING_STATUSES),
 Reservation.start_date <= return_date,
 Reservation.end_date >= pickup_date
 )
 query = query.filter(~Car.car_id.in_(booked_cars))
```

Backend admin check in car_admin.py. Every car change calls this first, so a customer cannot use the admin functions even if they find a way to call them:

```
def require_admin(session: Session, user_id: int):
 user = session.get(User, user_id)
 if not user or user.role != 'Administrator':
 raise PermissionError("Only administrators can manage cars.")
 return user
```

Technical issues and how we handled them

1. Hiding removed cars. Removing a car hides it instead of deleting it so past bookings are not lost. The database dump did not have a column for this, so we added an is_active column with add_is_active.sql. It has to be run once after the dump is loaded, so it should be added to the README setup steps.

2. Date overlap rule. A search that starts on the last day of an existing booking still counts as an overlap. We wrote the rule once in car_search.py and tested the edge cases (overlap, no overlap, and touching on the last day) so the browse page and search always agree.

3. Sample accounts. During testing we found that the sample accounts in the SQL dump (for example admin@drivenow.com) use placeholder password hashes, so they cannot sign in. For testing we created an administrator and a customer with the same hashing the app uses, and the customer account was made through the public Create Account form. The sample file should be updated with real hashes before the sprint review.

Items not built in Sprint 1

Booking, cancelling, and payments (RENT-01, RENT-02, PAY-01) were planned for Sprint 2, and the Book Now button is a placeholder. The My Account and My Bookings links show a message that the pages arrive in a later sprint. Three smaller items from the acceptance criteria are not built yet: editing a profile after sign-up (ACC-01), showing the number of seats and a car photo on each card (CAR-01,
which currently shows a drawn placeholder picture and has no seats field in the database), and an address field on the sign-up form.

5. Testing and Review

How we verified the prototype. We used two kinds of tests. First, the backend test script in the repository (test_backend.py) runs against an in-memory database and checks registration rules, hashing, sign-in, role access, active car listing, and the date overlap rule. It finished with ALL TESTS PASSED. Second, we ran scripted manual tests on the real application in which a script opens the actual Tkinter windows, fills in the fields, presses the buttons, and records what the screen and database show. These ran on a fresh MySQL 8.0 database loaded from the team's SQL files (with is_active added), with 7 active cars, 1 removed car, a test administrator, a test customer, and one booking entered by hand for 11/20/2026 to 11/23/2026. In total we ran 37 UI and database tests: 36 passed and 1 failed. The full log is in Appendix A and extra screenshots are in Appendix B.

How acceptance criteria were checked. Each acceptance criterion in the product backlog was matched to one or more tests, and a story is marked Done only when every criterion has a passing test.

Story	Acceptance criteria	Tests	Result
ACC-01	3 of 4 met. Sign-up works, duplicate emails are blocked, passwords are stored hashed. Editing a profile is not built yet.	ACC-01-T1 to T4	**Pending**
ACC-02	5 of 5 met. Correct and wrong sign-in, role-based landing pages, sign-out, and customers blocked from admin functions.	ACC-02-T1 to T8	**Pass / Done**
CAR-01	2 of 3 met. All active cars shown and removed cars hidden. Cards show make, model, type, year, and price, but not seats, and the photo is a placeholder drawing.	CAR-01-T1, T2	**Pending**
CAR-02	5 of 5 met. Dates, car type, price, and make filters work, booked cars are hidden, the no-results message and date rules show.	CAR-02-T1 to T11	**Pass / Done**
CAR-03	3 of 3 met. Clicking a car opens its details, the page shows its features and description, and Back to Search returns to the results.	CAR-03-T1 to T5	**Pass / Done**
ADMIN-01	4 of 4 met. An administrator can add, edit, and remove cars, edits show for customers, removed cars are hidden, and customers are blocked on the backend.	ADMIN-01-T1 to T6, ACC-02-T6	**Pass / Done**
SETUP.1	Sample data: the sample accounts in the SQL dump cannot sign in (placeholder password hashes). The cars and locations load correctly.	SETUP-T1	**Fail**

Review. Four of the six stories pass all their acceptance criteria. ACC-01 and CAR-01 are still pending because of the small gaps described above, and we plan to close them in the next sprint along with fixing the sample accounts.

Appendix A: Test Log

ID	Test	Expected result	Actual result	Pass/Fail
ACC-02 -T1	Click Sign In with both fields empty	Prompt to enter email and password	Please enter your email and password.	**PASS**
ACC-02 -T2	Wrong password for a real email	Invalid email or password.	Invalid email or password.	**PASS**
ACC-02 -T3	Unknown email, any password (message must be identical to wrong password)	Same message: Invalid email or password.	Invalid email or password.	**PASS**
SETUP-T1	Sign in with the seeded accounts in 'drivenow_db with data.sql' (admin@drivenow.com)	Sample accounts can sign in	Invalid email or password. (seed file stores placeholder 'mockhash' passwords)	**FAIL**
ACC-01 -T1	Submit registration with every field empty	'This field is required.' beside each required field (licence optional)	Required messages on: first_name, last_name, email, phone, password, confirm	**PASS**
ACC-01 -T2	Invalid email, 5-digit phone, 5-character password, mismatched confirm	Field-level messages for email, phone, password	{"email": "Enter a valid email address.", "phone": "Enter a valid 10-digit phone number.", "password": "Password must be at least 8 characters."}	**PASS**

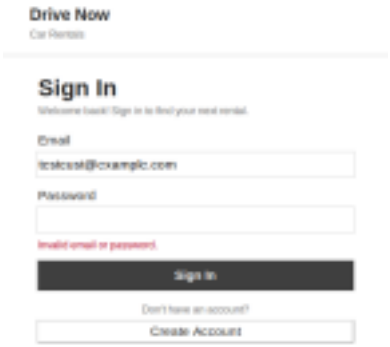
ACC-01 -T3	Register with an email that already exists	An account with this email already exists.	An account with this email already exists.	**PASS**
ACC-01 -T4	Register a valid new customer and inspect the database row	Account created as Customer; password stored as a hash	popup=Account created successfully! You can now sign in.; role=Customer; hash=scrypt:32768:8:1$BLpTN...	**PASS**
ACC-02 -T4	Customer signs in with correct credentials	Customer lands on the Browse Cars page	BrowseWindow	**PASS**
ACC-02 -T5	Customer tries to open the administrator page (role check)	Access denied	can_access('admin_dashboard') = False	**PASS**
ACC-02 -T6	Customer calls the admin add-car function directly (backend check)	Blocked on the backend	PermissionError: Only administrators can manage cars.	**PASS**
CAR-01 -T1	Open Browse Cars	Only active cars shown (7 active, 1 removed in DB)	7 cards: Toyota Corolla, Honda Civic, Hyundai Elantra, Toyota RAV4, Ford Escape, Subaru Outback, Subaru Outback; label '7 cars available'	**PASS**
CAR-01 -T2	Removed car is hidden from customers	Old Removed (is_active = 0) not shown	Old Removed in cards: False	**PASS**
CAR-02 -T1	Pick-up 12/20/2026, return 12/18/2026	Return date must be after pick-up date.	Return date must be after pick-up date.	**PASS**
CAR-02 -T2	Only a pick-up date entered	Enter both a pick-up date and a return date.	Enter both a pick-up date and a return date.	**PASS**
CAR-02 -T3	Minimum price 150 above maximum price 50	Minimum price cannot be higher than maximum price.	Minimum price cannot be higher than maximum price.	**PASS**

CAR-02 -T4	Pick-up date in the past	Pick-up date cannot be in the past.	Pick-up date cannot be in the past.	**PASS**
CAR-02 -T5	Search car type Van (no Vans in the data)	No cars match your search. Try different dates or filters.	No cars match your search. Try different dates or filters.	**PASS**

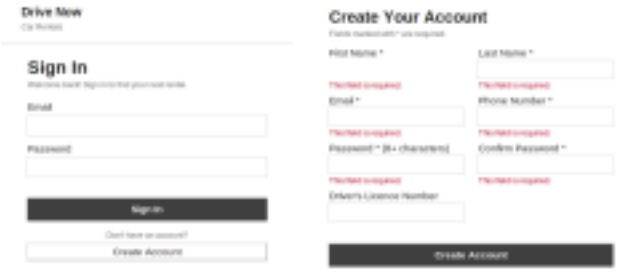
ID	Test	Expected result	Actual result	Pass/Fail
CAR-02 -T6	Corolla booked 11/20-11/23; search 11/22-11/25 (overlap)	Toyota Corolla hidden	6 cars available for 3 days; Corolla shown: False	**PASS**
CAR-02 -T7	Search 11/24-11/26 (no overlap)	Toyota Corolla shown	7 cars available for 2 days; Corolla shown: True	**PASS**
CAR-02 -T8	Search 11/23-11/24 (starts on the booking's last day)	Toyota Corolla hidden	Corolla shown: False	**PASS**
CAR-02 -T9	Filter by car type SUV	Only SUVs (4 in DB)	4 cards: Toyota RAV4, Ford Escape, Subaru Outback, Subaru Outback	**PASS**
CAR-02-T10	Filter by make Toyota	Only Toyota cars	Toyota Corolla, Toyota RAV4	**PASS**
CAR-02-T11	Set the maximum price filter to $60/day	Only cars at $60/day or less (Corolla, Civic, Elantra)	Toyota Corolla, Honda Civic, Hyundai Elantra	**PASS**
CAR-03 -T1	Click a car card on the browse page	That car's details page opens	opened=True; car=Toyota Corolla $45.00	**PASS**
CAR-03 -T2	Dates carried over from search (12/10-12/13)	3 days and total = rate x 3	3 days; total $135.00	**PASS**
CAR-03 -T3	On details page enter return date before pick-up	Return date must be after pick-up date.	Return date must be after pick-up date.	**PASS**

	date			
CAR-03 -T4	Click Book Now (placeholder in Sprint 1)	Message that booking arrives in Sprint 2; nothing is booked	Booking will be available in Sprint 2.	**PASS**
CAR-03 -T5	Click Back to Search	Returns to the browse page with the same results	details closed=True; cards=7	**PASS**
ACC-02 -T7	Click Logout and confirm	Session ends and sign-in screen returns	session user = None; root visible = True	**PASS**
ACC-02 -T8	Administrator signs in with correct credentials	Administrator lands on the admin car management page	AdminWindow	**PASS**
ADMIN -01-T 1	Open admin page	Table lists all cars including removed (8 in DB)	8 rows	**PASS**
ADMIN -01-T 2	Click Add Car with an empty form	Required-field messages (Status and Location default to Available / the only branch); nothing saved	errors on: make, model, year, car_type, daily_rate; cars before=8, after=8	**PASS**
ADMIN -01-T 3	Add a valid car (2024 Tesla Model 3, $95/day)	Car saved and shown in the table	in DB: True; status line: Added 2024 Tesla Model 3.	**PASS**
ADMIN -01-T 4	Select the new car and change its daily price to $99	Price updated in the database and in what customers see	daily_rate = 99.0; customer search shows True	**PASS**
ADMIN -01-T 5	Remove the new car	Hidden from customers (is_active = 0) but record kept, not deleted	record exists = True; is_active = False	**PASS**
ADMIN -01-T 6	Customer browses after admin removed Tesla Model 3	Tesla Model 3 not shown	Tesla shown: False	**PASS**

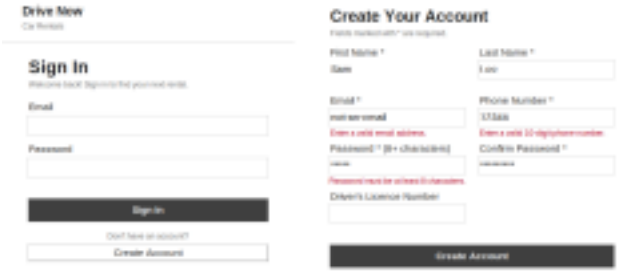
Appendix B: Additional Screenshots



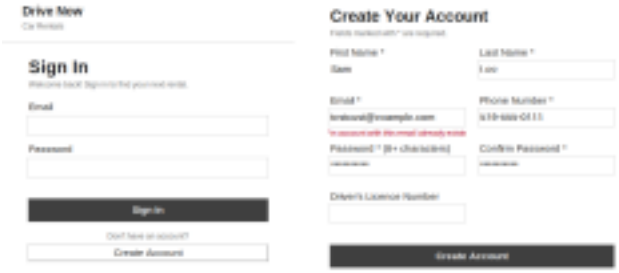
Sign-in with a wrong password shows the same message used for an unknown email.



Create an Account with every field empty.



Create an account with a bad email, short phone number, and short password.



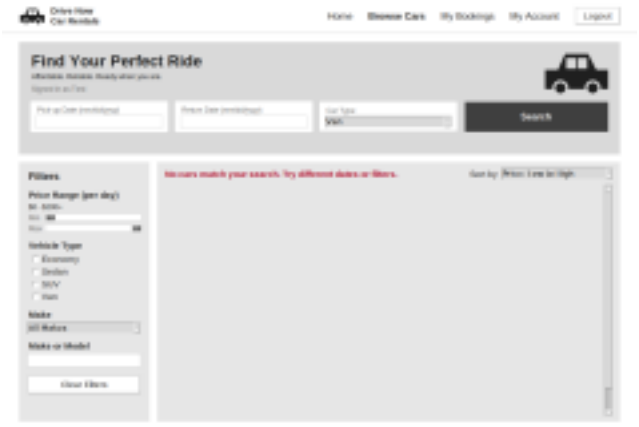
Create an Account with an email that already exists.



Search with the return date before the pick-up date.



Search with the minimum price above the maximum price.



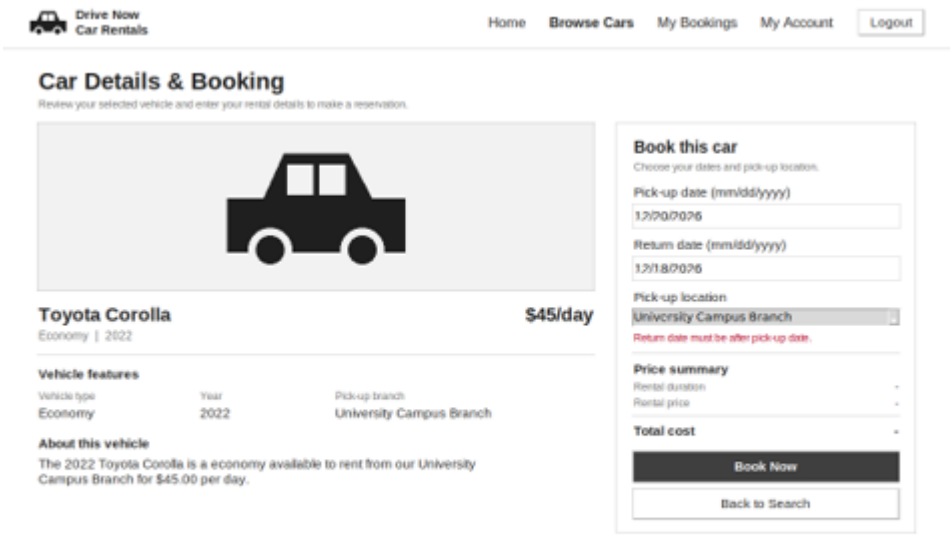
Search for a car type with no cars (Van) shows the no-results message.



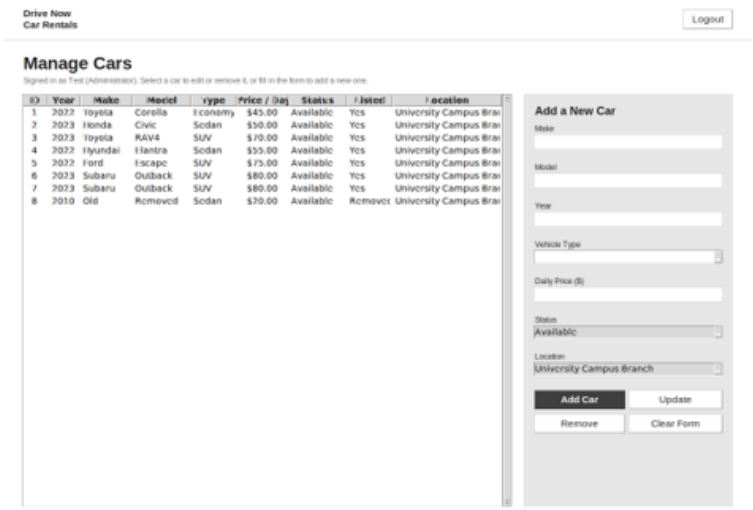
Car type filter set to SUV.



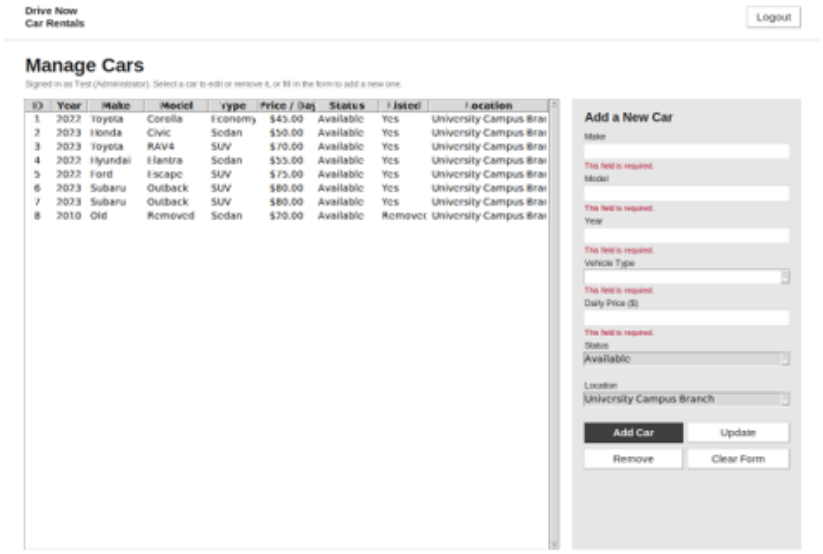
Maximum price filter set to $60 per day.



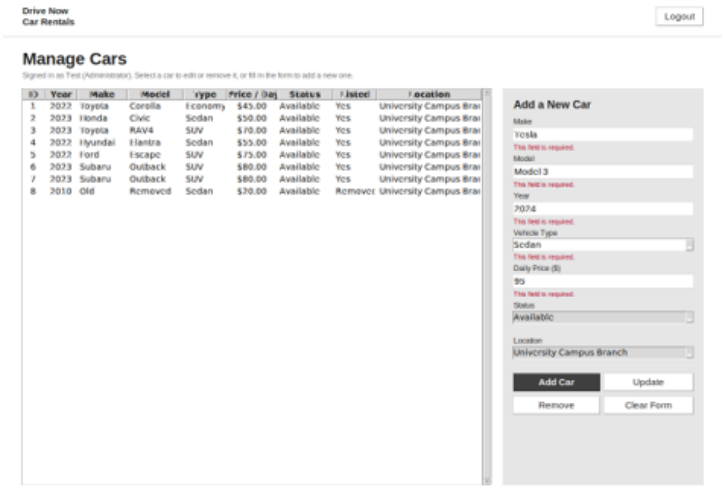
Car details page with a return date before the pick-up date.



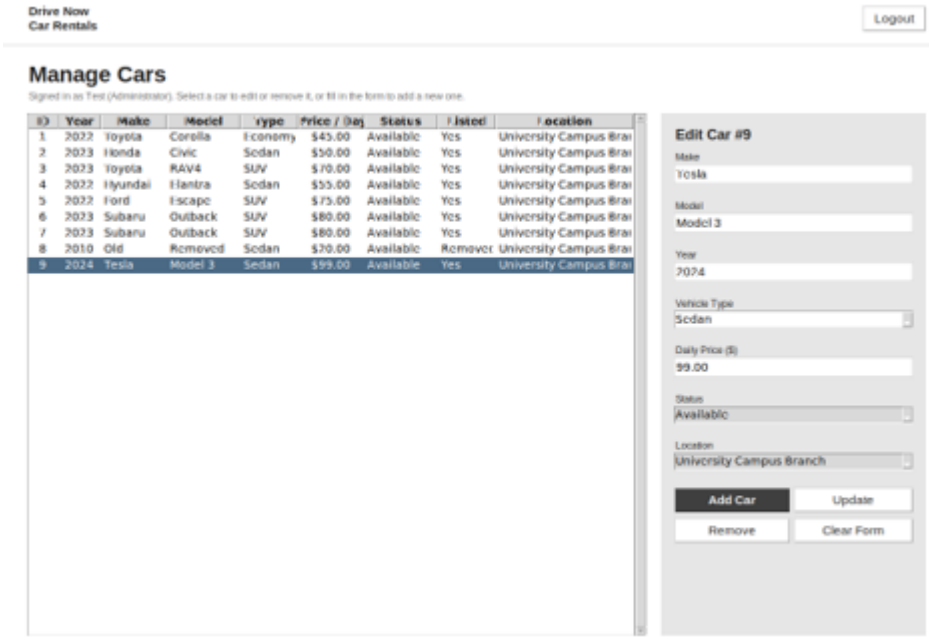
Administrator page after signing in.



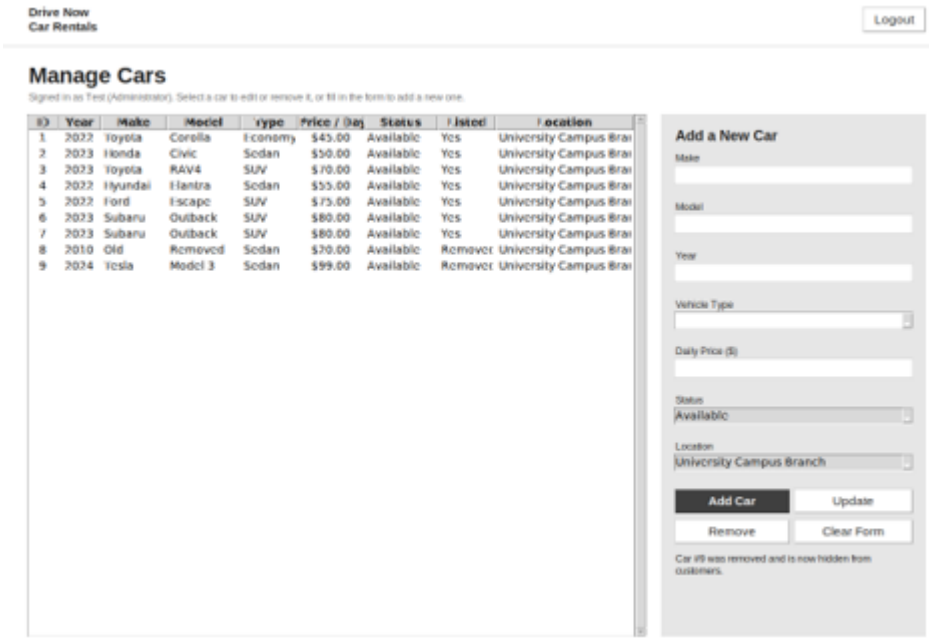
Add Car with an empty form shows the required-field messages.



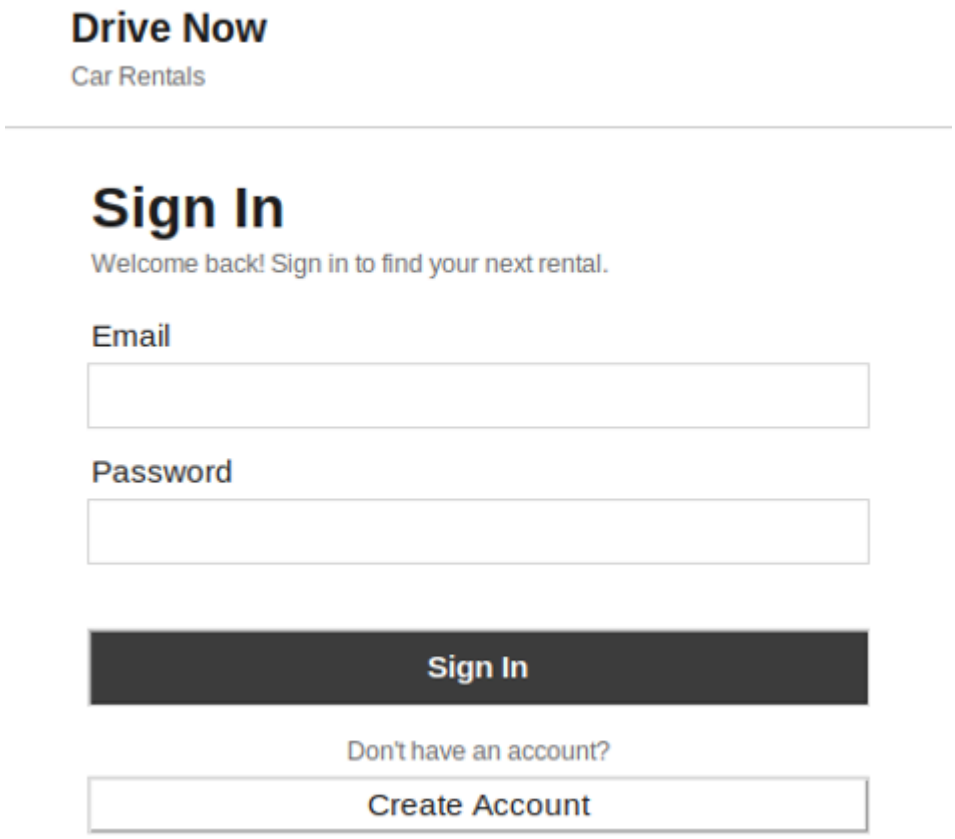
Add a Car form filled in with a new vehicle.



Daily price of the new car updated to $99.



New car removed. It stays in the table with Listed = Removed and is hidden from customers.



Back at the Sign In screen after logging out.